

Software Factory and CI/CD

Version 1, 2026-07-30

Software Factory and DevSpaces

Introduction

Welcome to this quick course on the **Software Factory and CI/CD**. This course is the **third** in a series of **six** courses about **Application Platform Delivery**:

- [Application Platform Delivery Foundations](#)
- [Software Factory & Developer Experience](#)
- [Software Factory and DevSpaces\(This course\)](#)
- [Software Factory and CICD](#)
- [Application Modernization](#)
- [Trusted Software Supply Chain \(TSSC\)](#)

What's in this course?

This training provides a comprehensive understanding of a modern Software Factory and its role in automating the software delivery lifecycle. Participants will learn how supporting services, Continuous Integration (CI), OpenShift Pipelines (Tekton), Continuous Delivery (CD), and promotion pipelines work together to build, validate, deploy, and promote applications across environments. By the end of the training, learners will understand how to implement standardized, automated, and GitOps-driven software delivery workflows on Red Hat OpenShift, enabling faster, more secure, and reliable application delivery.

Duration: 2.5 hours

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- .. (SME)
- .. (SME)
- Yogita Soni (Content Architect)
- Anna Swicegood (Learning Product Manager)

Objectives

On completing this course, you should be able to:

Based on the section index, here are concise learning objectives:

Learning Objectives

By the end of this module, you will be able to:

- Integrate Dev Spaces with GitLab to enable source code management and developer workflows.
- Create and use Dev Spaces workspaces for cloud-native application development.
- Understand how Continuous Integration (CI) pipelines are triggered and executed from developer workspaces.
- Deploy applications to OpenShift using automated delivery workflows.
- Explain the role of promotion pipelines in advancing applications across development, testing, and production environments.

Prerequisites

This course assumes that you have the following prior experience:

1. [Red Hat OpenShift Container Platform \(OCP\) Technical Overview](#) or Kubernetes Basics

Lab Environment

Click the [RHDP](#) link above, log in to RHDP, and click **Order** to request a new [Service Enablement: App Platform Software Factory](#). On the order page for the catalog item, do the following:

- **Activity:** Select **Practice/Enablement**
- **Purpose:** Select **Learning about the Product**
- **Salesforce ID:** If you have an opportunity ID, enter it into this field, otherwise select **Project**, and then enter "Learning about RHCL" in the text field. You will see a warning message that a Salesforce ID is required, but you can safely ignore it.
- **Features:** Select **Enable Let's Encrypt**
- **Region:** Select the AWS region closest to your location
- **User count:** **1**
- **Control Plane Count:** **1**
- **OpenShift Version:** **4.16**
- **Control Plane Instance Type:** **m6a.4xlarge**
- Select the checkbox at the bottom of the page to confirm that lab costs will be charged to your cost center, and click **Order**.



The lab environment will take an hour to provision. You can check the status and details of the OpenShift cluster from the **Services** page of RHDP. The classroom is available for 48 hours initially. You can extend the duration of classroom availability by clicking **Help > Get Technical Help** in RHDP, and then requesting an extension by opening a ServiceNow ticket with the RHDP team.

RHDP Portal Links

- [RedHat Associates](#)
- [RedHat Partners](#)

Software Factory and CI/CD

CI Pipeline

Overview

Create a Tekton CI pipeline that clones your application source, builds and tests it, retrieves registry credentials from Vault, and pushes the container image. Then wire up Tekton Triggers to fire the pipeline automatically when code is pushed to GitLab.

Pipeline Workspace and ServiceAccount

Create a workspace PVC and a ServiceAccount for the pipeline:

```
oc apply -f - <<'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pipeline-workspace-pvc
  namespace: etx-app-dev
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: pipeline
  namespace: etx-app-dev
EOF
```

Expected output:

```
persistentvolumeclaim/pipeline-workspace-pvc created
serviceaccount/pipeline created
```



If the ServiceAccount already exists (created during namespace setup), you may see **serviceaccount/pipeline configured** instead of **created**, along with a warning about a missing annotation. This is normal - **oc apply** automatically patches the annotation, and the resource is correctly configured.

Grant ServiceAccount Permissions

The pipeline ServiceAccount needs permission to push images and manage resources in the

namespace:

```
oc adm policy add-role-to-user edit \
  system:serviceaccount:etx-app-dev:pipeline \
  -n etx-app-dev
```

This grants the **edit** role, allowing the pipeline to create ImageStreams, Deployments, and other resources.

Retrieve Registry Credentials from Vault

Create a Tekton Task that authenticates to Vault using the Kubernetes auth method configured in the **Supporting Services** section and writes registry credentials to the workspace.

First, get the Vault URL from the cluster:

```
# Get Vault URL dynamically
VAULT_URL="https://$(oc get route -n vault -o jsonpath='{.items[0].spec.host}')"
echo "Using Vault URL: $VAULT_URL"
```

Then create the Task:

```
oc apply -f - <<EOF
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: vault-fetch-registry-creds
  namespace: etx-app-dev
spec:
  workspaces:
    - name: source
  results:
    - name: registry-server
    - name: registry-username
    - name: registry-password
  steps:
    - name: fetch
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      script: |
        #!/bin/sh
        set -e

        # Install jq
        microdnf install -y jq && microdnf clean all

        # Authenticate to Vault using ServiceAccount token
        VAULT_ADDR="${VAULT_URL}"
```

```

SA_TOKEN=\$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)

VAULT_TOKEN=\$(curl -s --request POST \
  "\${VAULT_ADDR}/v1/auth/kubernetes/login" \
  --data "{\"role\":\"pipeline\",\"jwt\":\"${SA_TOKEN}\"}" \
  | jq -r '.auth.client_token')

# Fetch registry credentials
CREDS=\$(curl -s --header "X-Vault-Token: \${VAULT_TOKEN}" \
  "\${VAULT_ADDR}/v1/secret/data/registry")

echo "\${CREDS}" | jq -r '.data.data.server' \
  | tr -d '\n' > \$(results.registry-server.path)
echo "\${CREDS}" | jq -r '.data.data.username' \
  | tr -d '\n' > \$(results.registry-username.path)
echo "\${CREDS}" | jq -r '.data.data.password' \
  | tr -d '\n' > \$(results.registry-password.path)

```

EOF

Create the Pipeline

Define the CI pipeline with four tasks: clone, build/test, fetch credentials, and build/push the container image:

```

oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: etx-app-base-app-ci
  namespace: etx-app-dev
spec:
  params:
    - name: git-url
      type: string
    - name: git-revision
      type: string
      default: main
    - name: image-name
      type: string
  workspaces:
    - name: shared-workspace
  tasks:
    - name: clone
      taskRef:
        resolver: cluster
        params:
          - name: kind
            value: task
          - name: name

```

```

        value: git-clone
      - name: namespace
        value: openshift-pipelines
    params:
      - name: URL
        value: $(params.git-url)
      - name: REVISION
        value: $(params.git-revision)
    workspaces:
      - name: output
        workspace: shared-workspace

- name: build-and-test
  runAfter: [clone]
  taskRef:
    resolver: cluster
    params:
      - name: kind
        value: task
      - name: name
        value: maven
      - name: namespace
        value: openshift-pipelines
    params:
      - name: GOALS
        value: ["package", "-DskipTests=false"]
    workspaces:
      - name: source
        workspace: shared-workspace

- name: fetch-creds
  runAfter: [clone]
  taskRef:
    name: vault-fetch-registry-creds
  workspaces:
    - name: source
      workspace: shared-workspace

- name: build-image
  runAfter: [build-and-test, fetch-creds]
  taskRef:
    resolver: cluster
    params:
      - name: kind
        value: task
      - name: name
        value: buildah
      - name: namespace
        value: openshift-pipelines
    params:
      - name: IMAGE

```

```

        value: $(params.image-name):$(params.git-revision)
      - name: DOCKERFILE
        value: Containerfile
    workspaces:
      - name: source
        workspace: shared-workspace
EOF

```



`git-clone`, `maven`, and `buildah` are standard Tasks installed in the `openshift-pipelines` namespace. This pipeline resolves them with the cluster resolver (replacing deprecated `ClusterTask` references). Run `oc get task -n openshift-pipelines` to verify they are available.

Test the Pipeline Manually

Run the pipeline once to verify it works before adding webhooks.

First, get the GitLab URL from the cluster:

```

# Get GitLab URL dynamically
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "Using GitLab URL: $GITLAB_URL"

```

Then create the PipelineRun:

```

oc create -f - <<EOF
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: etx-app-base-app-ci-
  namespace: etx-app-dev
spec:
  pipelineRef:
    name: etx-app-base-app-ci
  params:
    - name: git-url
      value: ${GITLAB_URL}/software-factory/etx_app_base_app
    - name: git-revision
      value: main
    - name: image-name
      value: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-app-
base-app
  workspaces:
    - name: shared-workspace
      persistentVolumeClaim:
        claimName: pipeline-workspace-pvc
EOF

```

```
taskRunTemplate:
  serviceAccountName: pipeline
EOF
```

Expected output:

```
pipelinerun.tekton.dev/etx-app-base-app-ci-xxxxx created
```

Monitor the PipelineRun:

```
# List all PipelineRuns
oc get pipelinerun -n etx-app-dev
```

You should see your PipelineRun with status **Running** and once complete it should be **Succeeded**.

Monitor Pipeline Execution

There are several ways to monitor the pipeline:

Option 1: Watch PipelineRun status (Recommended)

```
# Watch PipelineRun status in real-time (press Ctrl+C to exit)
oc get pipelinerun -n etx-app-dev -w
```

This shows the overall status of all PipelineRuns and updates in real-time.

Option 2: View individual Task logs

```
# Get the name of the latest PipelineRun
LATEST_PR=$(oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1].metadata.name}')

echo "PipelineRun: $LATEST_PR"

# List all TaskRuns for this PipelineRun
oc get taskrun -n etx-app-dev -l tekton.dev/pipelineRun=$LATEST_PR
```

To view logs of a specific TaskRun, use the name from the list above:

```
# View logs of a specific TaskRun (example: fetch-creds)
oc logs -n etx-app-dev -l tekton.dev/taskRun=etx-app-base-app-ci-xxxxx-fetch-creds -f
```

Or automatically get logs from a failed TaskRun:

```
# Get the latest PipelineRun
LATEST_PR=$(oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1].metadata.name}')

# Get the first failed TaskRun
FAILED_TASK=$(oc get taskrun -n etx-app-dev -l tekton.dev/pipelineRun=$LATEST_PR -o
jsonpath='{.items[?(@.status.conditions[0].reason=="Failed")].metadata.name}' | awk
'{print $1}')

if [ -n "$FAILED_TASK" ]; then
    echo "Viewing logs for failed TaskRun: $FAILED_TASK"
    oc logs -n etx-app-dev -l tekton.dev/taskRun=$FAILED_TASK
else
    echo "No failed TaskRuns found"
fi
```

Option 3: View logs of currently running pod

```
# Get the name of the latest PipelineRun
LATEST_PR=$(oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1].metadata.name}')

echo "Watching logs for PipelineRun: $LATEST_PR"

# Follow the logs of the currently running pod
oc logs -f $(oc get pods -n etx-app-dev -l tekton.dev/pipelineRun=$LATEST_PR --sort
-by=.metadata.creationTimestamp -o jsonpath='{.items[-1].metadata.name}')
```



Option 3 only shows logs from one Task at a time. When that Task completes, the command exits. Use Option 1 for continuous monitoring or Option 2 to view specific Task logs.

Watch the logs to confirm all tasks complete successfully.

Troubleshooting Common Issues

fetch-creds task fails with "parse error: Invalid numeric literal"

This error means the Vault response is not valid JSON. Common causes:

- **Vault is unavailable:** Check if Vault pods are running:

```
oc get pods -n vault
```

If Vault is down, wait for it to recover and retry the pipeline.

- **Vault authentication not configured:** Verify the Kubernetes auth method exists in Vault UI at the vault URL.
- **ServiceAccount lacks permissions:** Verify the pipeline ServiceAccount exists:

```
oc get serviceaccount pipeline -n etx-app-dev
```

Pipeline succeeds but image not found in registry

Verify the image was pushed:

```
oc get imagestream etx-app-base-app -n etx-app-dev
oc get imagestreamtag etx-app-base-app:main -n etx-app-dev
```

git-clone task fails with "Could not resolve host"

The GitLab URL is incorrect. Verify:

```
oc get route -n gitlab -l app=webservice -o jsonpath='{.items[0].spec.host}{"\n"}'
```

Recreate the PipelineRun with the correct GitLab URL.

Set Up Webhooks with Tekton Triggers

Create the trigger resources so the pipeline runs automatically when code is pushed to GitLab.

TriggerBinding

Maps fields from the GitLab webhook payload to pipeline parameters:

```
oc apply -f - <<'EOF'
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerBinding
metadata:
  name: gitlab-push-binding
  namespace: etx-app-dev
spec:
  params:
    - name: git-url
      value: $(body.repository.git_http_url)
    - name: git-revision
      value: $(body.checkout_sha)
EOF
```

TriggerTemplate

Creates a PipelineRun when triggered:

```
oc apply -f - <<'EOF'
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerTemplate
metadata:
  name: gitlab-push-template
  namespace: etx-app-dev
spec:
  params:
    - name: git-url
    - name: git-revision
  resourceTemplates:
    - apiVersion: tekton.dev/v1
      kind: PipelineRun
      metadata:
        generateName: etx-app-base-app-ci-
      spec:
        pipelineRef:
          name: etx-app-base-app-ci
        params:
          - name: git-url
            value: $(tt.params.git-url)
          - name: git-revision
            value: $(tt.params.git-revision)
          - name: image-name
            value: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-
app-base-app
        workspaces:
          - name: shared-workspace
            persistentVolumeClaim:
              claimName: pipeline-workspace-pvc
        taskRunTemplate:
          serviceAccountName: pipeline
EOF
```

EventListener

Exposes an HTTP endpoint that GitLab can send webhooks to:

```
oc apply -f - <<'EOF'
apiVersion: triggers.tekton.dev/v1beta1
kind: EventListener
metadata:
  name: gitlab-listener
  namespace: etx-app-dev
spec:
```

```
serviceAccountName: pipeline
triggers:
- name: gitlab-push
  bindings:
  - ref: gitlab-push-binding
  template:
    ref: gitlab-push-template
EOF
```

Expose the EventListener

Create a route with TLS so GitLab can reach the EventListener securely:

```
oc create route edge el-gitlab-listener \
--service=el-gitlab-listener \
--port=http-listener \
-n etx-app-dev
```



GitLab requires HTTPS for webhooks, so we create a route with edge TLS termination. If the route already exists, delete it first with `oc delete route el-gitlab-listener -n etx-app-dev` and recreate with TLS.

Get the webhook URL:

```
echo "https://$(oc get route el-gitlab-listener -n etx-app-dev -o
jsonpath='{.spec.host}')
```

Copy this URL for the next step.

Expected format:

```
https://el-gitlab-listener-etx-app-dev.apps.cluster-xxxxx.dynamic.redhatworkshops.io
```

Configure GitLab Webhook



Configuring webhooks in GitLab requires **Maintainer** or **Owner** role on the project. The `platform-developer` user does not have sufficient permissions. Use the `platform-admin` account or ask your instructor to grant you Maintainer access.

Step 1: Enable Webhooks to Local Network (REQUIRED)



GitLab blocks webhooks to local/internal networks by default for security (SSRF protection). Since the EventListener runs in the same cluster as GitLab, you **must** enable this feature or webhook creation will fail with "Invalid url given" error.

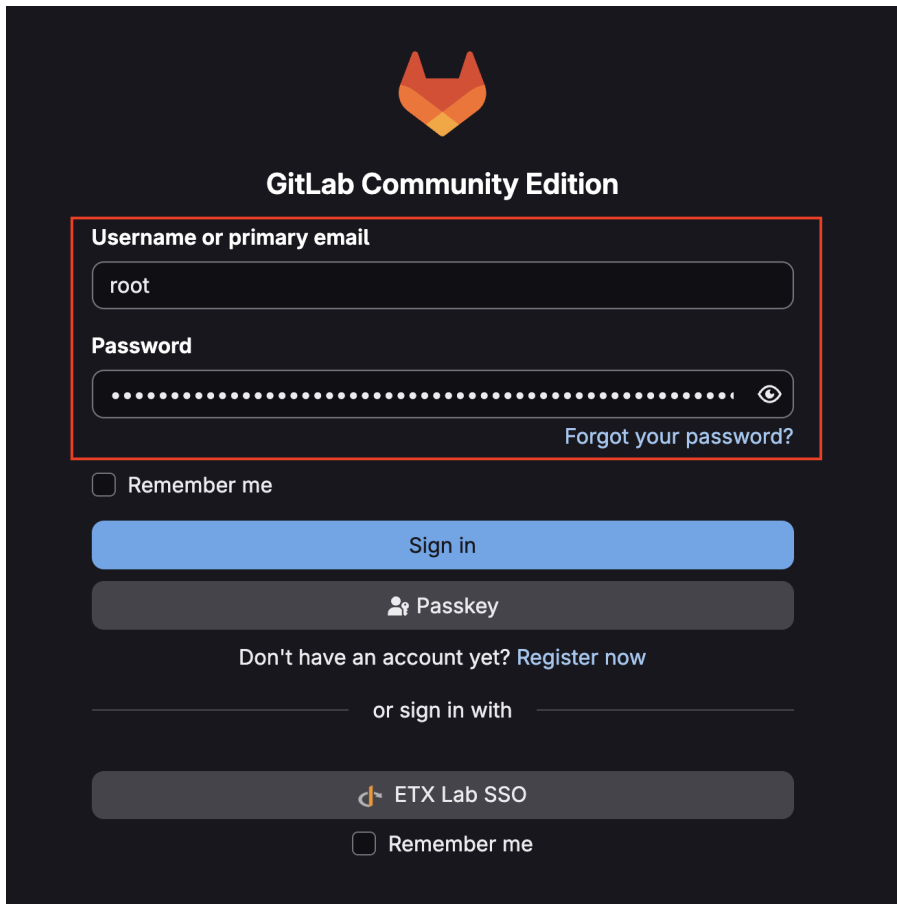
1. Get the GitLab Admin URLs and root password:

```
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o jsonpath='{.items[0].spec.host}')"
GITLAB_ROOT_PASSWORD=$(oc get secret -n gitlab gitlab-gitlab-initial-root-password -o jsonpath='{.data.password}' | base64 -d)

echo "GitLab URL: $GITLAB_URL"
echo "Admin Area: $GITLAB_URL/admin"
echo "Network Settings (Direct): $GITLAB_URL/admin/application_settings/network"
echo ""
echo "Root Username: root"
echo "Root Password: $GITLAB_ROOT_PASSWORD"
```

2. Log into GitLab as root administrator:

- Open the GitLab URL in your browser
- Click **Standard** tab (not ETX Lab SSO)



- **Username:** **root**
- **Password:** Use the password from the command above



The **root** user is the GitLab built-in administrator. The **platform-admin** user from Keycloak does not have admin privileges by default.

1. Access the Admin Area:

Option A - Direct URL (Recommended):

Open the admin URL directly (from the command above):

```
{gitlab_url}/admin/application_settings/network
```

This takes you directly to the Network settings page.

Option B - UI Navigation:

- After logging in, look for the **wrench icon**  in the top navigation bar (usually far right)
- OR click your **avatar/profile picture** (top right) → select **Admin Area**
- If you don't see these options, verify you're logged in as **platform-admin**

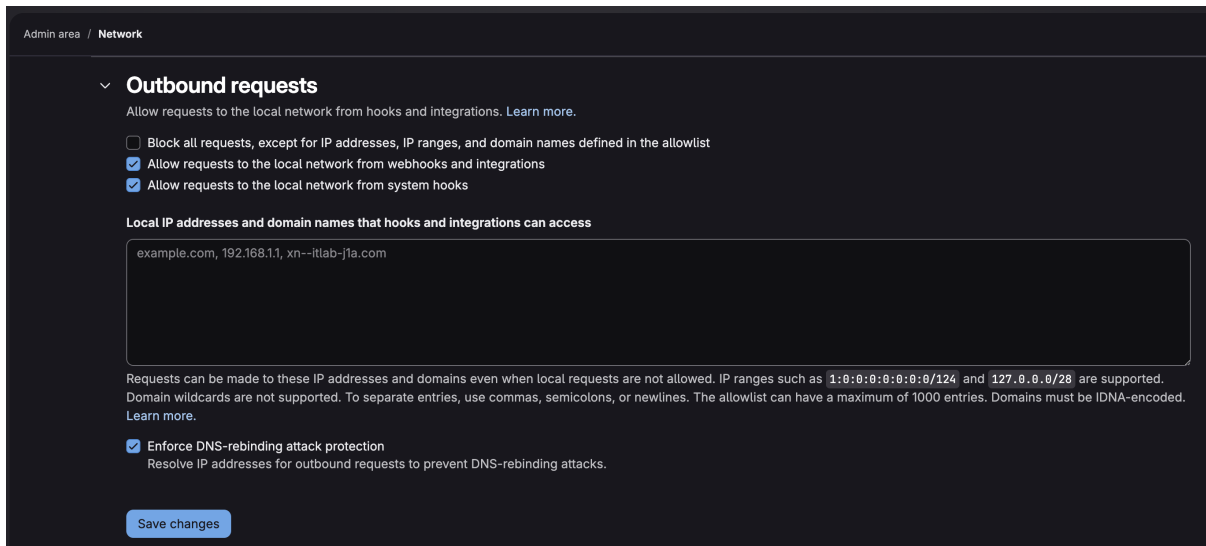
Then navigate:

- In the left sidebar, find **Settings** section
- Click **Settings** → **Network**

2. Scroll down the page and notice the **Outbound requests** section, expand the section.

3. Enable local network webhooks:

- Check ☒ **Allow requests to the local network from webhooks and integrations**
- Optionally check ☒ **Allow requests to the local network from system hooks**



Admin area / Network

▼ **Outbound requests**
Allow requests to the local network from hooks and integrations. [Learn more.](#)

☐ Block all requests, except for IP addresses, IP ranges, and domain names defined in the allowlist

☒ Allow requests to the local network from webhooks and integrations

☒ Allow requests to the local network from system hooks

Local IP addresses and domain names that hooks and integrations can access

example.com, 192.168.1.1, xn--[t]lab-j[a].com

Requests can be made to these IP addresses and domains even when local requests are not allowed. IP ranges such as `1:0:0:0:0:0/124` and `127.0.0.0/28` are supported. Domain wildcards are not supported. To separate entries, use commas, semicolons, or newlines. The allowlist can have a maximum of 1000 entries. Domains must be IDNA-encoded. [Learn more.](#)

☒ Enforce DNS-rebinding attack protection
Resolve IP addresses for outbound requests to prevent DNS-rebinding attacks.

[Save changes](#)

4. Click btn:[Save changes] at the bottom



Finding the Admin Area:

The Admin Area is only accessible to users with administrator privileges. When logged in as **root**, you should see:

- A wrench icon (🔧) in the top navigation bar
- "Admin Area" option when clicking your avatar

If you don't see these, verify:

- You're logged in as **root** (GitLab built-in admin)
- You used the Standard login (not ETX Lab SSO)
- You're using the correct password from the OpenShift secret

Why not platform-admin?

The **platform-admin** user from Keycloak is not a GitLab administrator by default. To grant admin privileges to **platform-admin**:

1. Log in as **root**
2. Go to menu:Admin Area[Users]
3. Find the **platform-admin** user
4. Click **Edit** → Check **Admin** → **Save changes**

Direct URL: You can always access admin settings directly at: `{gitlab_url}/admin/application_settings/network`

Verification: After saving, you should see a green checkmark or success message indicating "Allow requests to the local network from webhooks and integrations" is enabled.



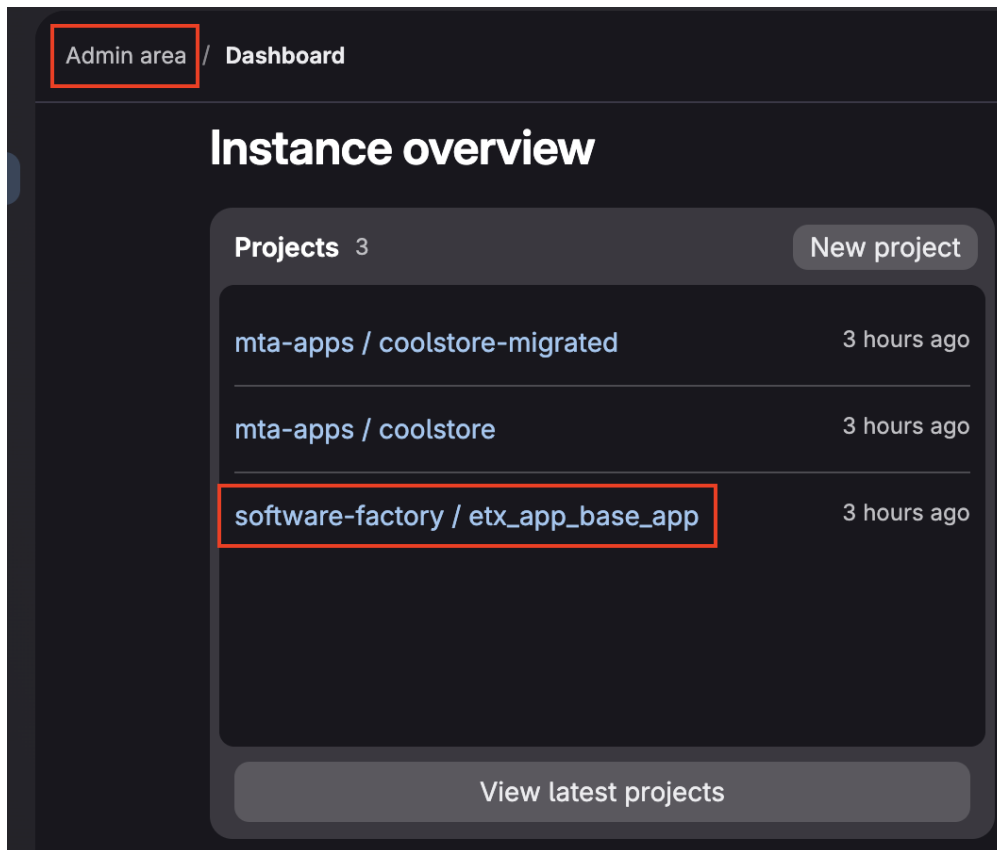
This setting is cluster-wide and affects all GitLab projects. In production environments, consider using external webhooks, GitLab Runner with allowedHosts, or more restrictive network policies.

Step 2: Add the Webhook to Your Repository

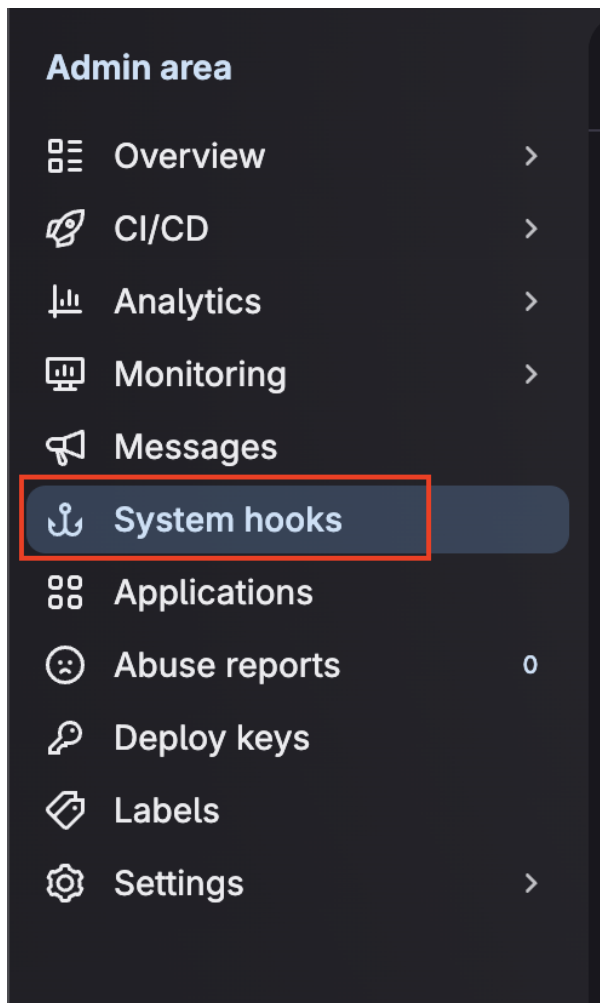


Configuring webhooks requires **Maintainer** or **Owner** role on the project (different from GitLab Admin). You can use **platform-admin** from Keycloak if it has Maintainer access to the project, or log in as **root**.

1. Navigate to the project, for this select **Admin area** → **Dashboard** → select **etx_app_base_app**



2. Go to menu: System hooks



3. Click btn:[Add new webhook]

4. Configure the webhook:

- **URL:** Paste the EventListener route URL:

```
echo "https://$(oc get route el-gitlab-listener -n etx-app-dev -o jsonpath='{.spec.host}')"

```

- **Trigger:** Check ☒ **Push events**
- **SSL verification:** Uncheck for lab environments with self-signed certificates

System hooks

Use webhooks to send notifications to external applications in response to events in a GitLab project or group.

System hooks 0

Name (optional)

Description (optional)

URL

https://el-gitlab-listener-etx-app-dev.apps.cluster-mp56f.dyn.redhatworkshops.io

URL must be percent-encoded if necessary.

Secret token

Use this token to validate received payloads.

Trigger

System hooks are triggered on sets of events like creating a project or adding an SSH key. You can also enable extra triggers, such as push events.

☒ **Repository update events**
URL is triggered when repository is updated

☒ **Push events**
URL is triggered for each branch updated to the repository

☐ **Tag push events**
URL is triggered when a new tag is pushed to the repository

☐ **Merge request events**
URL is triggered when a merge request is created, updated, or merged

SSL verification

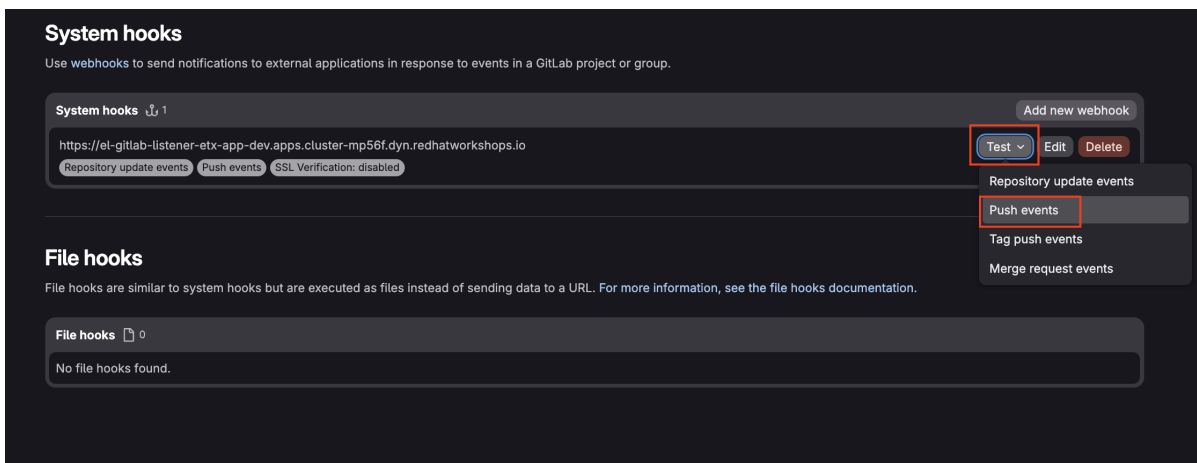
☐ **Enable SSL verification**

Add webhook Cancel

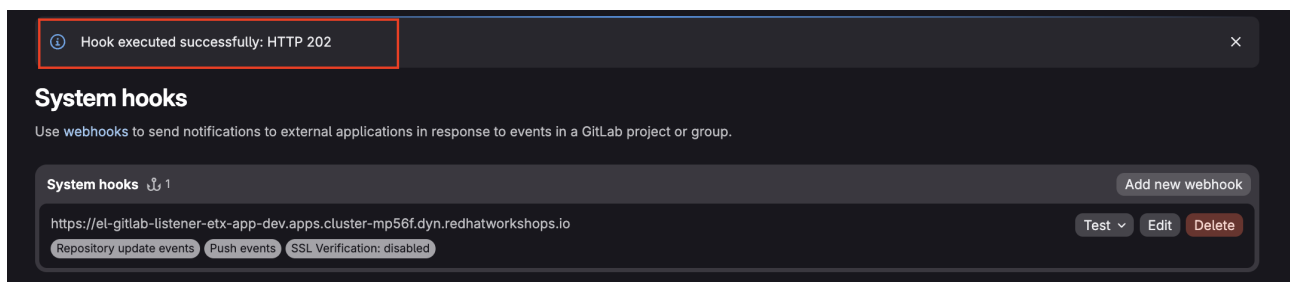
5. Click btn:[Add webhook]

6. Test the webhook:

- Find your webhook in the **System hooks** list
- Click btn:[Test] → **Push events**



- You should see "Hook executed successfully: HTTP 202" or similar



If you get "SSL certificate problem" error during testing, make sure "Enable SSL verification" is unchecked in the webhook configuration.

Troubleshooting Webhook Issues

Error: "Invalid url given"

This error occurs when GitLab blocks webhooks to local/internal networks. Solutions:

1. Enable local network webhooks (required):

- Go to menu: Admin Area[Settings > Network > Outbound requests]
- Check "Allow requests to the local network from webhooks and integrations"
- Save changes and try again

2. Verify the URL format:

- Must start with `https://` (not `http://`)
- Run: `echo "https://$(oc get route el-gitlab-listener -n etx-app-dev -o jsonpath='{.spec.host}')"`

3. Test EventListener is responding:

```
curl -I https://$(oc get route el-gitlab-listener -n etx-app-dev -o jsonpath='{.spec.host}')
```

Expected: HTTP 400 Bad Request (this is normal - it expects a webhook payload)

Error: "SSL certificate problem"

The cluster uses self-signed certificates. In GitLab webhook settings:

- Uncheck "Enable SSL verification"
- This is safe for lab environments

Test event fails with error

Check the EventListener pod is running:

```
oc get pods -n etx-app-dev | grep el-gitlab-listener
oc logs -n etx-app-dev -l eventlistener=gitlab-listener
```

Verify the Pipeline

Push a change to your repository and confirm the pipeline triggers:

```
# Make a trivial change
echo "# CI/CD test" >> README.md
git add README.md
git commit -m "Test webhook trigger"
git push origin feature/add-devfile
```

Check for new pipeline runs:

```
oc get pipelinerun -n etx-app-dev
```

You should see a new **etx-app-base-app-ci-*** PipelineRun triggered by the webhook.

Follow the latest run logs:

```
# Get the name of the latest PipelineRun
LATEST_PR=$(oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1].metadata.name}')

echo "Following logs for: $LATEST_PR"

# Follow the logs
oc logs -f $(oc get pods -n etx-app-dev -l tekton.dev/pipelineRun=$LATEST_PR --sort
-by=.metadata.creationTimestamp -o jsonpath='{.items[-1].metadata.name}') -n etx-app-
dev
```

Verify the image was pushed:

```
oc get imagestream etx-app-base-app -n etx-app-dev
```

You should see the image tagged with the commit SHA.

Quick Verification

Verify all components are working:

```
# Check pipeline exists
oc get pipeline etx-app-base-app-ci -n etx-app-dev

# Check EventListener is running
oc get pods -n etx-app-dev | grep el-gitlab-listener

# Check webhook URL is exposed
oc get route el-gitlab-listener -n etx-app-dev

# Check recent pipeline runs
oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp | tail -5
```



Common Issues:

- **Pipeline fails at clone:** Check Git credentials/authentication
- **Build fails:** Verify Maven build works locally first
- **Push fails:** Confirm ServiceAccount has **edit** role
- **Webhook not triggering:** Verify EventListener pod is Running and route is accessible

Summary

You now have:

- Tekton CI pipeline with Vault secrets integration
- Automated builds triggered by GitLab webhooks
- Container images pushed to OpenShift internal registry
- Maven-based Java/Quarkus build and test automation

Next Steps

Proceed to [Application Deployment with Argo CD](#).

Application Deployment

Overview

Deploy your own Argo CD instance to manage application deployments using a GitOps workflow. The CI pipeline from the previous section pushes container images; Argo CD watches a GitOps repository and syncs Kubernetes manifests to the cluster.



You will deploy your **own** Argo CD instance in the **etx-app-dev** namespace. The pre-deployed OpenShift GitOps operator in **openshift-gitops** is used for infrastructure automation only and is separate from your application deployment workflows.

Install OpenShift GitOps Operator

The OpenShift GitOps operator is already installed cluster-wide from the infrastructure setup. Verify it's available:

```
oc get csv -n openshift-operators | grep gitops
```

You should see **openshift-gitops-operator** with phase **Succeeded**.

Configure Keycloak OIDC for Argo CD

The ETX platform uses a shared Keycloak OIDC client (**argocd**) for all ArgoCD instances. This provides centralized authentication with the same credentials across GitLab, Quay, Vault, and ArgoCD.

Create ArgoCD OIDC Secret

The OIDC client **argocd** already exists in Keycloak realm **etx-lab**. Create a secret with the shared credentials:

```
# Get Keycloak URL (etx-sso route)
KEYCLOAK_URL="https://etx-sso.apps.${oc get route -n keycloak -o
jsonpath='{.items[0].spec.host}' | sed 's/sso\.apps\.//'}"
```

```
# Use the shared argocd client secret from etx-gitops
CLIENT_SECRET="changeme"
```

```
oc create secret generic argocd-oidc-secret -n etx-app-dev \
  --from-literal=clientSecret=${CLIENT_SECRET}
```

```
# Add labels to help ArgoCD operator recognize the secret
oc label secret argocd-oidc-secret -n etx-app-dev \
  app.kubernetes.io/name=argocd-oidc-secret \
```

```
app.kubernetes.io/part-of=argocd
```

```
echo "Keycloak issuer: ${KEYCLOAK_URL}/realms/etx-lab"
```



The ArgoCD OIDC client is shared across all ArgoCD instances in the ETX platform. Do not create a new client in Keycloak. Use the existing **argocd** client with the provided secret.

Deploy Your Argo CD Instance

Create an Argo CD instance in your namespace for managing application deployments.

First, get the cluster domain dynamically:

```
# Get cluster domain from existing Keycloak route
CLUSTER_DOMAIN="$(oc get route -n keycloak -o jsonpath='{.items[0].spec.host}' | sed
's/sso\.apps\.//')"
```

```
echo "Cluster domain: ${CLUSTER_DOMAIN}"
```

Then create the ArgoCD instance with OIDC configuration:

```
CLUSTER_DOMAIN="$(oc get route -n keycloak -o jsonpath='{.items[0].spec.host}' | sed
's/sso\.apps\.//')"
```

```
cat <<EOF | oc apply -f -
apiVersion: argoproj.io/v1beta1
kind: ArgoCD
metadata:
  name: argocd
  namespace: etx-app-dev
spec:
  server:
    autoscale:
      enabled: false
    grpc:
      ingress:
        enabled: false
    ingress:
      enabled: false
    insecure: true
  resources:
    limits:
      cpu: 500m
      memory: 256Mi
    requests:
      cpu: 125m
      memory: 128Mi
```

```

route:
  enabled: true
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
service:
  type: ClusterIP
grafana:
  enabled: false
  ingress:
    enabled: false
  route:
    enabled: false
prometheus:
  enabled: false
  ingress:
    enabled: false
  route:
    enabled: false
initialSSHKnownHosts: {}
rbac:
  defaultPolicy: 'role:readonly'
  policy: |
    g, platform-admin, role:admin
    g, platform-developer, role:admin
  scopes: '[groups, preferred_username]'
extraConfig:
  oidc.config: |
    name: ETX Keycloak
    issuer: https://etx-sso.apps.${CLUSTER_DOMAIN}/realms/etx-lab
    clientID: argocd
    clientSecret: \${argocd-oidc-secret:clientSecret}
    requestedScopes:
      - openid
    requestedIDTokenClaims:
      groups:
        essential: true
    logoutURL: https://etx-sso.apps.${CLUSTER_DOMAIN}/realms/etx-lab/protocol/openid-connect/logout
repo:
  resources:
    limits:
      cpu: 1000m
      memory: 1Gi
    requests:
      cpu: 250m
      memory: 256Mi
resourceExclusions: |
  - apiGroups:
    - tekton.dev
  clusters:

```

```

- '*'
kinds:
- TaskRun
- PipelineRun
controller:
  processors: {}
  resources:
    limits:
      cpu: 2000m
      memory: 2Gi
    requests:
      cpu: 250m
      memory: 1Gi
ha:
  enabled: false
  resources:
    limits:
      cpu: 500m
      memory: 256Mi
    requests:
      cpu: 250m
      memory: 128Mi
EOF

```

Expected output:

```
argocd.argoproj.io/argocd created
```



The `server.insecure: true` setting configures Argo CD to serve HTTP traffic internally while the OpenShift route handles TLS termination. This is required to prevent redirect loops (`ERR_TOO_MANY_REDIRECTS`) when using edge TLS termination.



The ArgoCD CR includes `extraConfig` with OIDC configuration for ETX Keycloak integration. This provides centralized authentication using the same credentials as GitLab, Quay, and Vault. The `argocd-oidc-secret` created earlier is referenced automatically.

Wait for Argo CD to be ready (this may take 3-5 minutes):

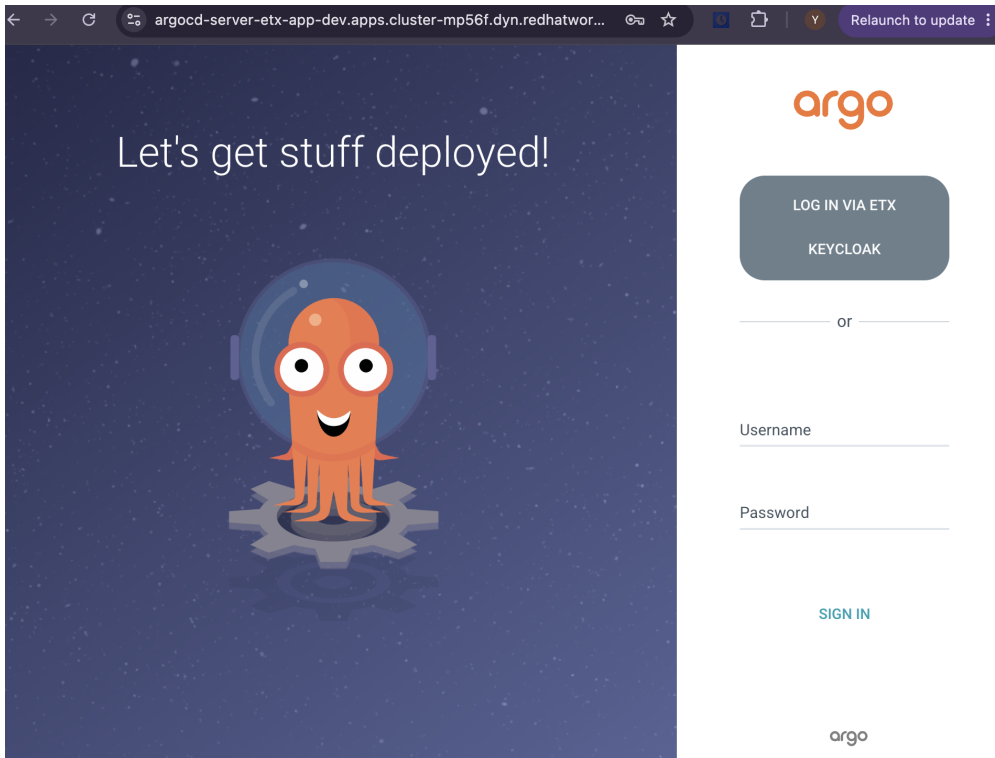
```
oc get pods -n etx-app-dev -w
```

Press Ctrl+C when you see all `argocd-*` pods showing `1/1 Ready`.

Verify Route Creation

The ArgoCD operator should automatically create a route. Verify it exists:

```
oc get route argocd-server -n etx-app-dev
```



If the route doesn't exist, create it manually:

```
oc create route edge argocd-server \
  --service=argocd-server \
  --port=http \
  --insecure-policy=Redirect \
  -n etx-app-dev
```

Access the Argo CD Dashboard

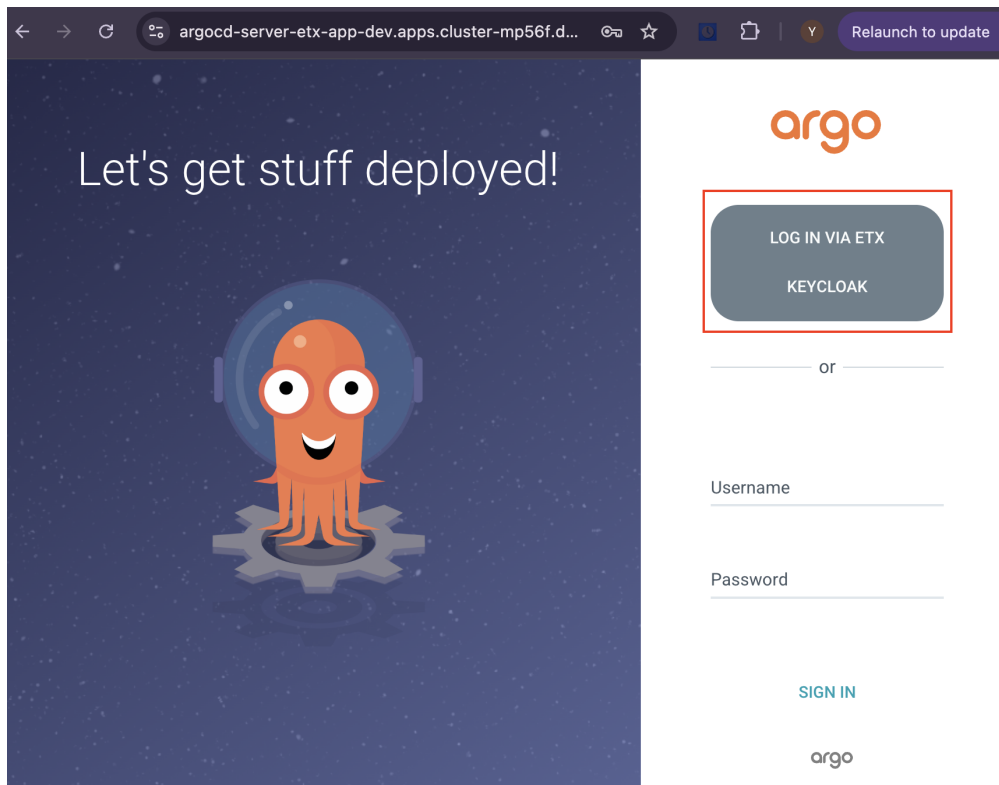
Get the Argo CD dashboard URL:

```
echo "https://$(oc get route argocd-server -n etx-app-dev -o jsonpath='{.spec.host}')
```

You can log into Argo CD using either the admin account or Keycloak SSO.

Option 1: ETX Keycloak SSO (Recommended)

1. Open the Argo CD URL in your browser
2. Select the **ETX Keycloak login** option



3. Log in with your ETX credentials:

- `platform-admin` / `openshift` (admin access)
- `platform-developer` / `openshift` (admin access)

4. After successful authentication, you'll be redirected back to the Argo CD dashboard



The OIDC integration provides automatic redirect to Keycloak for a streamlined login experience. No need to click a separate login button.

Option 2: Admin Account

Get the default admin password:

```
oc get secret argocd-cluster -n etx-app-dev \
-o jsonpath='{.data.admin\.password}' | base64 -d && echo
```

Open the Argo CD URL and log in with: * Username: `admin` * Password: (from above command)



Using Keycloak SSO is recommended as it provides centralized authentication with the same credentials used for GitLab, Quay, and Vault.

Troubleshooting: ERR_TOO_MANY_REDIRECTS

If you get `ERR_TOO_MANY_REDIRECTS` when accessing the Argo CD dashboard:

Cause: The Argo CD server is not configured for insecure mode behind a TLS-terminating proxy.

Solution:

1. Update the ArgoCD CR to add `server.insecure: true`:

```
oc patch argocd argocd -n etx-app-dev --type=merge -p
'{"spec":{"server":{"insecure":true}}}'
```

2. Wait for the argocd-server pod to restart (this happens automatically):

```
oc get pods -n etx-app-dev -l app.kubernetes.io/name=argocd-server -w
```

Press Ctrl+C when you see the new pod showing **1/1 Running**.

3. Verify the configuration was applied:

```
oc get argocd argocd -n etx-app-dev -o jsonpath='{.spec.server.insecure}'
```

Should output: **true**

4. Clear your browser cache or use incognito/private mode to avoid cached redirects
5. Try accessing the dashboard again:

```
echo "https://$(oc get route argocd-server -n etx-app-dev -o
jsonpath='{.spec.host}')
```



If you still see the error after following these steps, try a different browser or clear all site data for the ArgoCD URL. The browser may have cached the redirect loop.

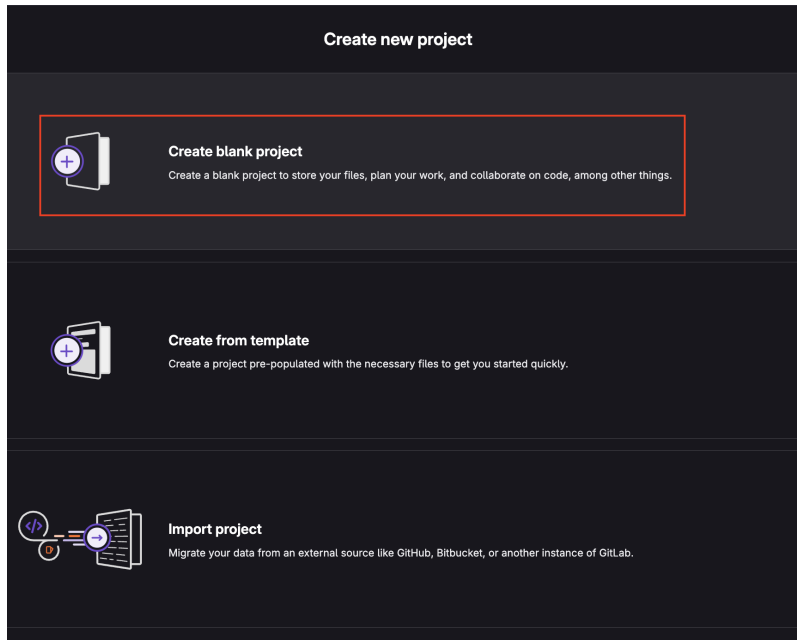
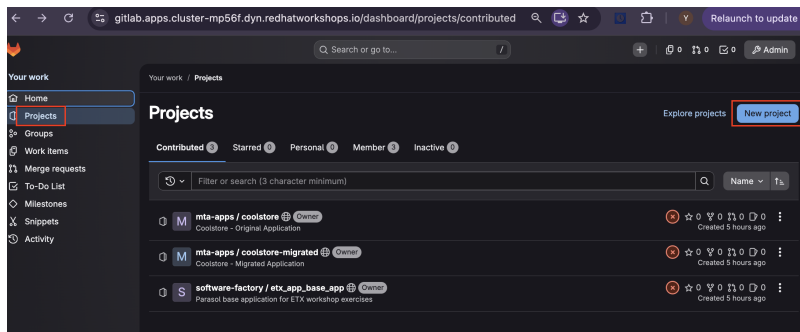
Prepare the GitOps Repository

Create a GitOps repository in GitLab with the following structure. This repository holds the Kubernetes manifests that Argo CD will sync:

1. Navigate to your openshift console terminal and log into GitLab (get the URL first):

```
# Get GitLab URL
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "GitLab URL: $GITLAB_URL"
```

2. Create a new project: **etx-app-gitops** in the software-factory group



Create blank project

Create a blank project to store your files, plan your work, and collaborate on code, among other things.

Project name

etx-app-gitops

Must start with a lowercase or uppercase letter, digit, emoji, or underscore. Can also contain dots, pluses, dashes, or spaces.

Project URL

https://gitlab.apps.cluster-mp56f.dyn.redhatworkshops.io/

Project slug

software-factory / etx-app-gitops

Visibility Level ?

☐ Private
Project access must be granted explicitly to each user. If this project is part of a group, access is granted to members of the group.

☐ Internal
The project can be accessed by any logged in user except external users.

☒ Public
The project can be accessed without any authentication.

Project Configuration

☒ Initialize repository with a README
Allows you to immediately clone this project's repository. Skip this if you plan to push up an existing repository.

☐ Enable Static Application Security Testing (SAST)
Analyze your source code for known security vulnerabilities. [Learn more.](#)

☐ Enable Secret Detection
Scan your code for secrets and credentials to prevent unauthorized access. [Learn more.](#)

Create project Cancel

3. Clone it locally:

```
git clone ${GITLAB_URL}/software-factory/etx-app-gitops.git
cd etx-app-gitops
```

We will create the following directory structure in the GitOps repository:

```
etx-app-gitops/
├── environments/
│   ├── dev/
│   │   ├── namespace.yaml
│   │   ├── deployment.yaml
│   │   ├── service.yaml
│   │   └── route.yaml
│   └── stage/
│       ├── namespace.yaml
│       ├── deployment.yaml
│       ├── service.yaml
│       └── route.yaml
└── README.md
```

Dev Environment Manifests

Create `environments/dev/namespace.yaml`:

```
mkdir -p environments/dev
cat > environments/dev/namespace.yaml <<'EOF'
apiVersion: v1
kind: Namespace
metadata:
  name: etx-app-dev
EOF
```

Create `environments/dev/deployment.yaml`:

```
cat > environments/dev/deployment.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etx-app-base-app
  namespace: etx-app-dev
  labels:
    app: etx-app-base-app
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true" # ①
spec:
  replicas: 1
```

```

selector:
  matchLabels:
    app: etx-app-base-app
template:
  metadata:
    labels:
      app: etx-app-base-app
    annotations:
      instrumentation.opentelemetry.io/inject-java: "true" # ①
  spec:
    containers:
      - name: etx-app-base-app
        image: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-app-
base-app:latest ②
        ports:
          - containerPort: 8080
        resources:
          requests:
            cpu: 250m
            memory: 512Mi
          limits:
            cpu: "1"
            memory: 1Gi
EOF

```

① Enables OpenTelemetry auto-instrumentation. The Instrumentation CR created in the **Supporting Services** section will inject the Java agent automatically.

② The CI pipeline updates this image tag after a successful build.

Create `environments/dev/service.yaml`:

```

cat > environments/dev/service.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: etx-app-base-app
  namespace: etx-app-dev
spec:
  selector:
    app: etx-app-base-app
  ports:
    - port: 8080
      targetPort: 8080
EOF

```

Create `environments/dev/route.yaml`:

```

cat > environments/dev/route.yaml <<'EOF'
apiVersion: route.openshift.io/v1

```

```
kind: Route
metadata:
  name: etx-app-base-app
  namespace: etx-app-dev
spec:
  to:
    kind: Service
    name: etx-app-base-app
  port:
    targetPort: 8080
  tls:
    termination: edge
EOF
```

Commit and push the dev environment:

```
git config --global user.email "platform-developer@example.com"
git config --global user.name "Platform Developer"
GITLAB_ROOT_PASSWORD=$(oc get secret -n gitlab gitlab-gitlab-initial-root-password -o
jsonpath='{.data.password}' | base64 -d)
echo "Root Password: $GITLAB_ROOT_PASSWORD"
echo "Root Username: root"

git add environments/dev/
git commit -m "feat: add dev environment manifests"
git push origin main
```

Create the Argo CD Application

Grant Argo CD permission to manage resources in the **etx-app-dev** namespace:

```
oc adm policy add-role-to-user admin \
  system:serviceaccount:etx-app-dev:argocd-application-controller \
  -n etx-app-dev
```

Create an Argo CD Application that watches the **dev** environment.

First, get the GitLab URL:

```
# Get GitLab URL dynamically
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "Using GitLab URL: $GITLAB_URL"
```

Then create the Application:

```

oc apply -f - <<EOF
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: etx-app-dev
  namespace: etx-app-dev
spec:
  project: default
  source:
    repoURL: ${GITLAB_URL}/software-factory/etx-app-gitops.git
    targetRevision: main
    path: environments/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: etx-app-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=false
EOF

```



`CreateNamespace=false` because the namespace already exists.

Verify the Deployment

Check that Argo CD has synced the application:

```

# Argo CD application status
oc get application etx-app-dev -n etx-app-dev \
  -o jsonpath='{.status.sync.status}'

```

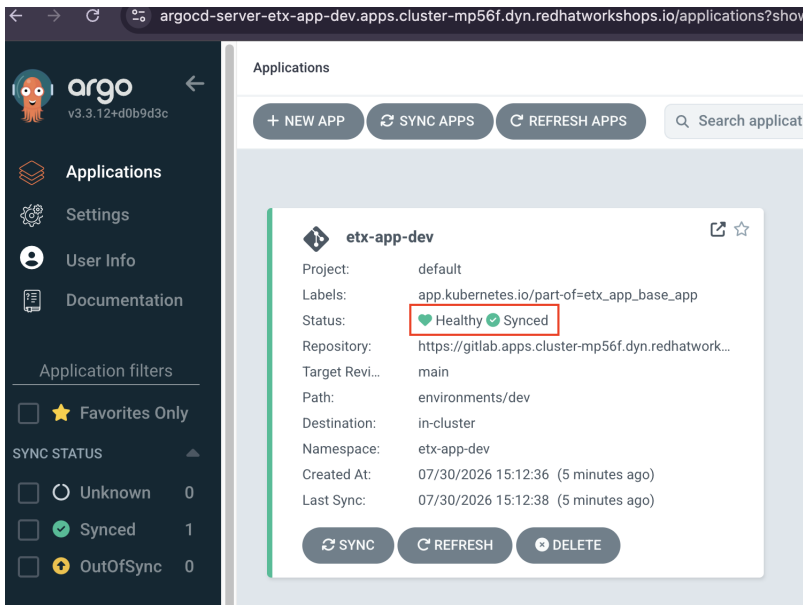
The output should be **Synced**.

You can even navigate to argocd server and check the status of application

```

# Argo CD console URL
oc get route argocd-server -n etx-app-dev -o jsonpath='{.spec.host}'

```



Verify the application is running:

```
oc get pods -n etx-app-dev -l app=etx-app-base-app
oc get route etx-app-base-app -n etx-app-dev -o jsonpath='{.spec.host}{"\n"}'
```

Open the route URL in your browser to confirm the application is accessible.

[app] | *app*

Click on the application to view more details

[app deploy] | *app_deploy*

Verify OpenTelemetry Auto-Instrumentation

Check that the Java agent was injected into the application pod:

```
oc get pod -n etx-app-dev -l app=etx-app-base-app \
-o jsonpath='{.items[0].spec.initContainers[*].name}{"\n"}'
```

You should see an **opentelemetry-auto-instrumentation** init container.

Verify traces are being collected by checking the OTEL collector logs:

```
oc logs -n etx-app-dev -l app.kubernetes.io/name=otel-collector-collector --tail=50
```

You should see trace data in the logs.

Integrate CI and GitOps

The CI pipeline needs to update the image tag in the GitOps repository after a successful build. Add a final task to the pipeline that commits the new image tag:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: update-gitops-repo
  namespace: etx-app-dev
spec:
  params:
    - name: gitops-repo-url
      type: string
    - name: image-tag
      type: string
    - name: environment
      type: string
      default: dev
  workspaces:
    - name: source
  steps:
    - name: update-image
      image: registry.access.redhat.com/ubi9/ubi:latest
      script: |
        #!/bin/bash
        set -e

        # Install git
        dnf install -y git && dnf clean all

        REPO_URL="$(params.gitops-repo-url)"
        GITLAB_HOST=$(echo "$REPO_URL" | sed -E 's|https?://([^\/]+)/.*|\1|')

        git config --global url."https://etx-platform-
bot:openshift@${GITLAB_HOST}/".insteadOf "https://${GITLAB_HOST}/"

        git clone "$REPO_URL" /tmp/gitops
        cd /tmp/gitops/environments/${params.environment}

        # Update the image tag in the deployment manifest
        sed -i "s|image:.*etx-app-base-app:.*|image: image-registry.openshift-image-
registry.svc:5000/etx-app-dev/etx-app-base-app:${params.image-tag}|" deployment.yaml

        git config user.email "pipeline@openshift.local"
        git config user.name "Tekton Pipeline"
        git add .
        git commit -m "Update etx-app-base-app image to ${params.image-tag}"
        git push
```

EOF

Update the pipeline to include this task at the end.

Get the GitLab URL first, then update the pipeline:

```
# Get GitLab URL dynamically
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"

# Add to your pipeline YAML under spec.tasks
```

Add the **update-gitops** task to the pipeline spec.

```
oc edit pipeline etx-app-base-app-ci -n etx-app-dev
```

Task definition to add, please note that it is suppose to be added under **tasks** section:

```
tasks:
- name: update-gitops
  params:
  - name: gitops-repo-url
    value: SUBSTITUTE-GITLAB-URL-HERE/software-factory/etx-app-gitops.git
  - name: image-tag
    value: $(params.git-revision)
  - name: environment
    value: dev
  runAfter:
  - build-image
  taskRef:
    kind: Task
    name: update-gitops-repo
  workspaces:
  - name: source
    workspace: shared-workspace
```



Use shell variable expansion `${GITLAB_URL}` when creating the pipeline with a heredoc, or replace with the actual URL when editing the YAML file.

When the pipeline pushes the updated manifest, Argo CD detects the change and automatically syncs the new image to the cluster.

Summary

You have completed the Application Deployment exercise. Your application now has:

- Your own Argo CD instance for GitOps deployments
- GitOps repository structure (dev and stage environments)
- Automated sync with self-healing enabled
- OpenTelemetry auto-instrumentation for distributed tracing
- Integration between CI pipeline and GitOps repository

Promotion Pipeline

Overview

In the previous exercises, you set up a CI pipeline that builds container images and deploys them to the dev environment automatically. Now you'll create a promotion pipeline that moves tested images from dev to staging on the secondary cluster.

This promotion pipeline:

- Copies container images from the internal registry to Quay (for cross-cluster access)
- Updates the GitOps repository with the new image tag for staging
- Triggers Argo CD to sync the staging environment on the secondary cluster

Prerequisites

Before starting this exercise, ensure you have completed:

- [Application CI Pipeline](#)
- [Application Deployment](#)
- Secondary cluster enrollment (instructions below)

You should have:

- A working CI pipeline that builds and pushes images
- Argo CD managing both dev and stage environments
- Access to Quay via Keycloak SSO (`platform-admin` or `platform-developer` / `openshift`)
- Runtime cluster provisioned and enrolled

Secondary Cluster Enrollment

The promotion pipeline requires two clusters:

- **Factory Cluster** (where you are now) - Development environment and ArgoCD
- **Runtime Cluster** - Staging and production environments

You need to enroll the runtime cluster with ArgoCD on the factory cluster.

Automated Enrollment

Download and run the enrollment script:

```
curl -o enroll-secondary-cluster.sh \
https://raw.githubusercontent.com/rh-etx-app-
```

```
platform/etx_app_tooling/main/scripts/enroll-secondary-cluster.sh
```

```
chmod +x enroll-secondary-cluster.sh
```

Edit the configuration section with your cluster credentials (from RHDP emails), substitute XXXXX, {openshift_cluster_admin_password} with your lab GUID and lab password :

```
export RUNTIME_API="https://api.cluster-xxxxx.dyn.redhatworkshops.io:6443"
export RUNTIME_USER="admin"
export RUNTIME_PASSWORD="{openshift_cluster_admin_password}"

export FACTORY_API="https://api.cluster-xxxxx.dyn.redhatworkshops.io:6443"
export FACTORY_USER="admin"
export FACTORY_PASSWORD="{openshift_cluster_admin_password}"
```

Run the script:

```
./enroll-secondary-cluster.sh
```

You will notice output as below

```
[STEP] Step 1: Preparing runtime cluster for ArgoCD management
[INFO] Logging into runtime cluster: https://api.cluster-
mp56f.dyn.redhatworkshops.io:6443
[INFO] Creating ArgoCD manager ServiceAccount...
serviceaccount/argocd-manager created
[INFO] Granting cluster-admin permissions to ArgoCD manager...
clusterrolebinding.rbac.authorization.k8s.io/argocd-manager-binding created
[INFO] Generating registration token (valid for 8760h)...
[INFO] Creating staging namespaces on runtime cluster...
namespace/etx-app-staging created
namespace/etx-app-prod created
[STEP] Step 1.5: Deploying supporting services on runtime cluster
[INFO] Installing AMQ Streams (Kafka) operator...
Warning: resource subscriptions/amq-streams is missing the kubernetes.io/last-
applied-configuration annotation which is required by oc apply. oc apply should only
be used on resources created declaratively by either oc create --save-config or oc
apply. The missing annotation will be patched automatically.
subscription.operators.coreos.com/amq-streams configured
[INFO] Installing OpenShift Pipelines (Tekton) operator...
Warning: resource subscriptions/openshift-pipelines-operator-rh is missing the
kubernetes.io/last-applied-configuration annotation which is required by oc
apply. oc apply should only be used on resources created declaratively by either oc
create --save-config or oc apply. The missing annotation will be patched
automatically.
subscription.operators.coreos.com/openshift-pipelines-operator-rh configured
[INFO] Installing Red Hat build of OpenTelemetry operator...
```

```

subscription.operators.coreos.com/opentelemetry-product created
[INFO] Waiting for operators to be ready (90s)...
[INFO] Deploying PostgreSQL database in etx-app-staging...
secret/parasol-db-secret created
service/parasol-db created
deployment.apps/parasol-db created
[INFO] Deploying Kafka cluster in etx-app-staging...
kafkanodepool.kafka.strimzi.io/kafka created
kafka.kafka.strimzi.io/parasol-kafka created
[INFO] Creating Kafka topic 'intake' in etx-app-staging...
kafkatopic.kafka.strimzi.io/intake created
[INFO] Deploying OpenTelemetry Collector in etx-app-staging...
opentelemetrycollector.opentelemetry.io/otel-collector created
[INFO] Configuring OpenTelemetry auto-instrumentation for Java in etx-app-staging...
instrumentation.opentelemetry.io/java-instrumentation created
[INFO] Supporting services deployment initiated (will take 3-4 minutes to be fully
ready)
[STEP] Step 2: Registering runtime cluster in factory ArgoCD
[INFO] Logging into factory cluster: https://api.cluster-
mp56f.dyn.redhatworkshops.io:6443
[INFO] Creating ArgoCD cluster secret...
secret/cluster-staging created
[STEP] Step 3: Validating enrollment
[INFO] Cluster secret verified: cluster-staging

```

```

\033[0;32m=====
Enrollment Complete
=====
\033[0m

```

```

Factory cluster: https://api.cluster-mp56f.dyn.redhatworkshops.io:6443
Runtime cluster: https://api.cluster-mp56f.dyn.redhatworkshops.io:6443
ArgoCD namespace: etx-app-dev
Cluster name: staging

```

Supporting services deployed:

- PostgreSQL (parasol-db) in etx-app-staging
- Kafka cluster (parasol-kafka) in etx-app-staging
- Kafka topic (intake)
- OpenTelemetry Collector (otel-collector) in etx-app-staging
- OpenTelemetry auto-instrumentation (java-instrumentation)
- AMQ Streams operator
- OpenShift Pipelines operator
- Red Hat build of OpenTelemetry operator

Next steps:

1. Verify cluster in ArgoCD UI:
 - Navigate to Settings > Clusters
 - Look for 'staging' cluster
2. Verify supporting services are ready:


```
oc get pods -n etx-app-staging
```

```
oc get kafka -n etx-app-staging
oc get kafkatopic -n etx-app-staging
oc get opentelemetrycollector -n etx-app-staging
oc get instrumentation -n etx-app-staging
```

3. Create Applications targeting runtime cluster:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
spec:
  destination:
    name: staging
    namespace: etx-app-staging
```

4. Test deployment to runtime cluster

Documentation: https://github.com/rh-etx-app-platform/etx_app_tooling/blob/main/docs/enrollment.md

Verify Enrollment

Check cluster secret exists:

```
oc get secret -n etx-app-dev -l argocd.argoproj.io/secret-type=cluster -o json | \
jq -r '.items[] | .metadata.name + " - " + (.data.name | @base64d)'
```

You should see secrets for:

```
argocd-default-cluster-config - in-cluster
cluster-staging - staging
```

You can now proceed with the promotion pipeline.

Understanding the Promotion Flow

The promotion workflow follows these steps:

Table 1. Promotion Sequence

Step	Component	Action
1	Developer	Approves promotion (manual trigger)
2	GitLab	Triggers promotion pipeline
3	Promotion Pipeline	Pulls image from internal registry by SHA
4	Promotion Pipeline	Pushes image to Quay with 'stable' tag
5	Promotion Pipeline	Updates <code>environments/stage/deployment.yaml</code> in GitOps repo

Step	Component	Action
6	Argo CD	Detects change in GitOps repository
7	Argo CD	Syncs new image to staging cluster

Key points:

- **Dev environment** uses images from internal registry (fast, local)
- **Staging environment** uses images from Quay (cross-cluster accessible)
- **Promotion is explicit** - you decide when code is ready for staging
- **GitOps ensures** staging deployments are version-controlled

Configure Quay for Image Promotion

The promotion pipeline pushes images to Quay at `etx-app/etx-app-base-app` and reads credentials from Vault at `secret/registry/quay`.

Quay authenticates interactive users through Keycloak SSO, so local `quayadmin` credentials cannot be used for automated pushes. You will create the organization and repository the pipeline expects, create a robot account for non-interactive authentication, and store the robot credentials in Vault.

Create the Quay Organization and Repository

1. Open [Quay](#) or click the **Quay** tab in your Showroom
2. Sign in with Keycloak SSO using `platform-admin` or `platform-developer / openshift`
3. Remove any "-" in the name to avoid syntax error and select **Confirm Username**

Confirm Username

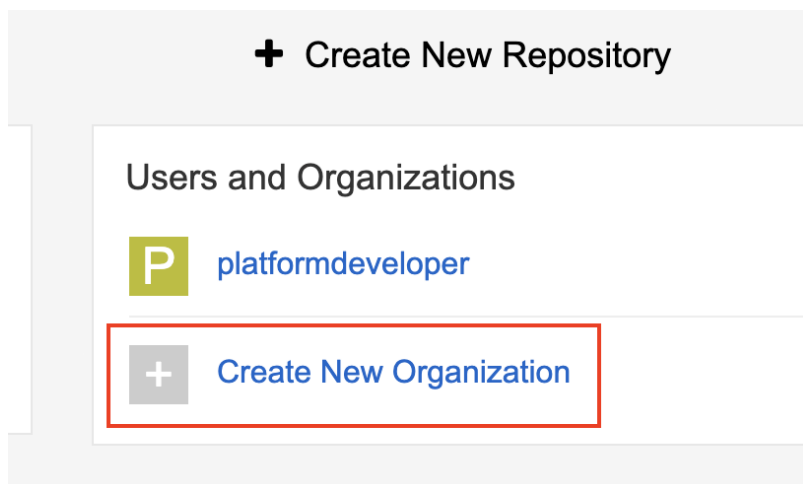
The username `platform-developer` was automatically generated to conform to the Docker CLI guidelines for use as a namespace in Red Hat Quay .

Please confirm the selected username or enter a different username below:

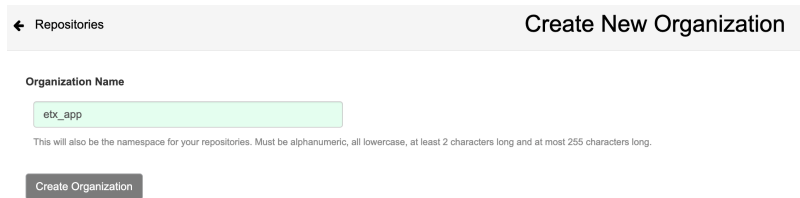
Confirm Username

✔ Username valid

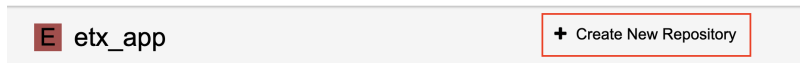
4. Click btn:[Create New Organization]



5. Set the organization name to `etx_app` and click btn:[Create Organization]



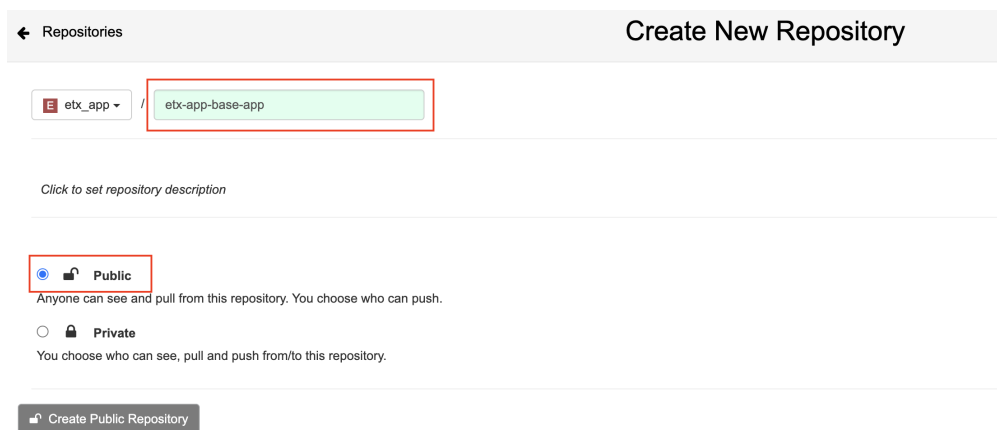
6. Inside the `etx_app` organization, click btn:[Create New Repository]



7. Configure the repository:

- **Repository Name:** `etx-app-base-app`
- **Repository Visibility:** Public

8. Click btn:[Create Public Repository]



The organization and repository names must match the destination used later in the promotion pipeline: `etx_app/etx-app-base-app`.

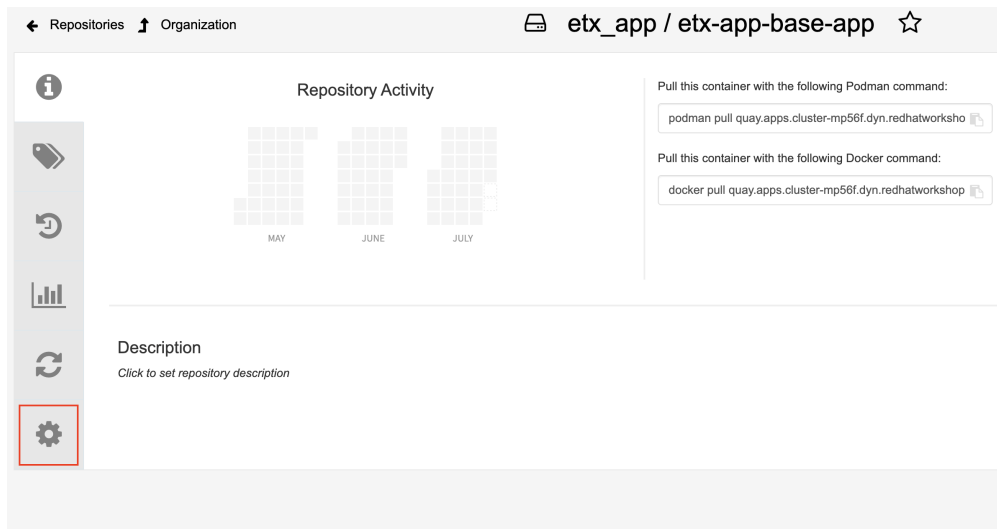


This Quay instance disables anonymous pulls (`FEATURE_ANONYMOUS_ACCESS: false`). Even with a public repository, the staging cluster must authenticate using an image pull secret.

Create a Robot Account

Robot accounts provide token-based credentials for CI/CD pipelines without requiring interactive SSO login.

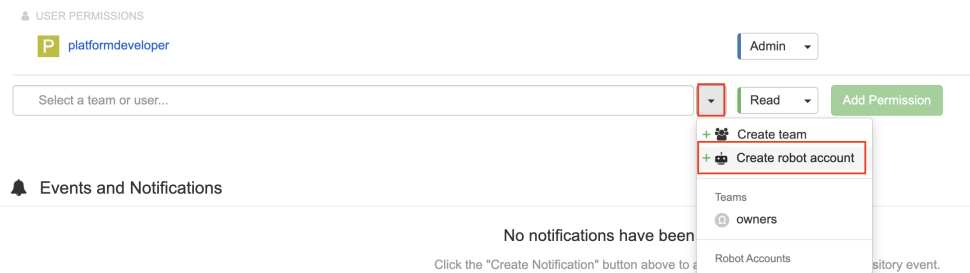
1. In the `etx_app` organization, click the **Settings** gear on the left panel



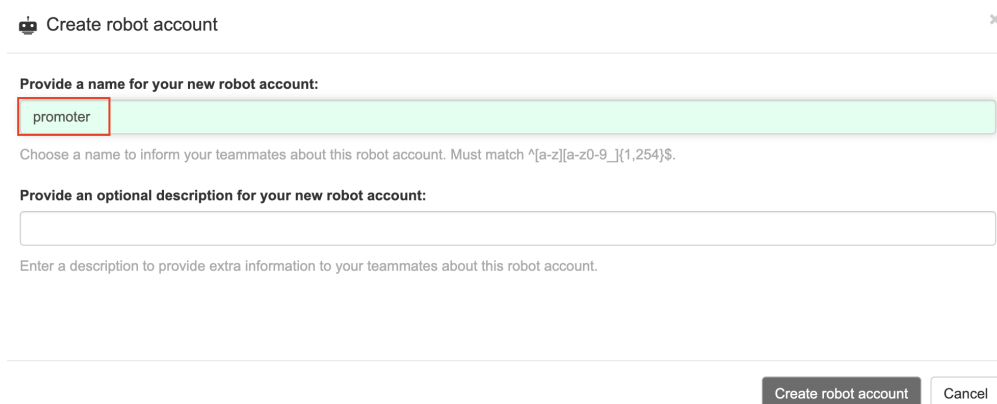
2. Under **User and Robot Permissions** click the down arrow next to the **Select a team or user...** box, click btn:[Create Robot Account]

Repository Settings

User and Robot Permissions



3. Name the robot **promoter** and click btn:[Create Robot Account]



4. Grant the robot **Write** permission by change the **Read** dropdown to **Write** and click btn:[Add Permission]

Repository Settings

User and Robot Permissions

USER PERMISSIONS

platformdeveloper

Admin

etx_app+promoter

Write

Add Permission

5. Click the newly created robot account **etx-app+promoter**
6. When Quay displays the robot credentials, copy both values:
 - **Username:** **etx-app+promoter**
 - **Token:** the generated token

Robot Account

Kubernetes Secret

rkt Configuration

Podman Login

Docker Login

Docker Configuration

Mesos Credentials

Credentials for etx_app+promoter

Username & Robot Account:

etx_app+promoter	
4QQXPH4WI0CZOQG2VQMENUG064MOKGHX3YQPX24ZL5PBQEK6H0VK2Y5XCTSRTZS	

Regenerate Token:

Click the link below to regenerate the token for this robot. Note that **all existing logins** of this robot account will become invalid.

[Regenerate Token](#)

Create Vault Secret with Robot Credentials

Enable a KV secrets engine in Vault and store the robot account credentials so the promotion pipeline can authenticate to Quay.

First, confirm the Quay hostname (this is the **server** value stored in Vault):

```
oc get route -n quay -l quay-component=quay-app-route -o jsonpath='{.items[0].spec.host}' && echo
```

Next, get the Vault root token:

```
oc get secret etx-vault-bootstrap -n vault -o jsonpath='{.data.root-token}' | base64 -d && echo
```

Enable the KV Secrets Engine

1. Open [Vault](#) or click the **Vault** tab in your Showroom
2. Log in with the **Token** method using the token above
3. From the Dashboard click **secret/** under the **Secret engines**
4. From the **secret/** secrets engine, click btn:[Create secret]

5. Set the path to `registry/quay`
6. Add the following secret data:
 - **server:** the Quay hostname from the command above (no `https://` prefix)
 - **username:** `etx-app+promoter`
 - **password:** the robot token you copied earlier
7. Click btn:[Save]

The promotion pipeline tasks you create next will fetch these credentials at runtime via Kubernetes auth to Vault.

Multi-Cluster Promotion to Stage

Create Stage Environment Manifests

Create stage environment manifests in your GitOps repository:

```
cd ~/workshop/etx-app-gitops
mkdir -p environments/stage
```

Copy the dev manifests as a template:

```
cp environments/dev/*.yaml environments/stage/
```

Update `environments/stage/deployment.yaml` to use the `etx-app-staging` namespace and Quay registry.

First, get the Quay URL:

```
# Get Quay URL dynamically
QUAY_URL=$(oc get route -n quay -l quay-component=quay-app-route -o
jsonpath='{.items[0].spec.host}')
echo "Using Quay URL: $QUAY_URL"
```

Create Quay Image Pull Secret on Staging

The staging cluster must authenticate to Quay when pulling images. Create a docker-registry secret in `etx-app-staging` on the **runtime** cluster using the robot account credentials.

Login to the runtime cluster (credentials from RHDP):

```
oc login $RUNTIME_API \
-u $RUNTIME_USER -p $RUNTIME_PASSWORD --insecure-skip-tls-verify
```

Replace `$ROBOT_TOKEN` with the `etx-app+promoter` token:

```
oc create secret docker-registry quay-pull-secret \
  -n etx-app-staging \
  --docker-server=$QUAY_URL \
  --docker-username='etx-app+promoter' \
  --docker-password=$ROBOT_TOKEN
```

Switch back to the factory cluster before continuing with the GitOps manifests:

```
oc login $FACTORY_API -u $FACTORY_USER -p $FACTORY_PASSWORD --insecure-skip-tls-verify
```

Create Application Deployment Manifest

```
cat > environments/stage/deployment.yaml <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etx-app-base-app
  namespace: etx-app-staging
  labels:
    app: etx-app-base-app
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true"
spec:
  replicas: 2
  selector:
    matchLabels:
      app: etx-app-base-app
  template:
    metadata:
      labels:
        app: etx-app-base-app
      annotations:
        instrumentation.opentelemetry.io/inject-java: "true"
    spec:
      imagePullSecrets:
        - name: quay-pull-secret
      containers:
        - name: etx-app-base-app
          image: ${QUAY_URL}/etx-app/etx-app-base-app:stable
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: 500m
              memory: 1Gi
            limits:
```

```

      cpu: "2"
      memory: 2Gi
    env:
      - name: POSTGRESQL_HOST
        value: "parasol-db.etx-app-staging.svc.cluster.local"
      - name: POSTGRESQL_DATABASE
        value: "parasol"
      - name: POSTGRESQL_USER
        value: "parasol"
      - name: POSTGRESQL_PASSWORD
        value: "parasol"
      - name: QUARKUS_HIBERNATE_ORM_DATABASE_GENERATION
        value: update
EOF

```

Key differences from dev:

- **Namespace:** `etx-app-staging` instead of `etx-app-dev`
- **Replicas:** Increased to 2 for higher availability
- **Image:** Uses Quay registry for cross-cluster image distribution instead of internal registry
- **imagePullSecrets:** References `quay-pull-secret` so the staging cluster can authenticate to Quay

Update `environments/stage/service.yaml` and `environments/stage/route.yaml` to use the `etx-app-staging` namespace:

```

sed -i 's/namespace: etx-app-dev/namespace: etx-app-staging/' \
  environments/stage/service.yaml \
  environments/stage/route.yaml

```

Verify the changes:

```

grep namespace environments/stage/*.yaml

```

Commit and push:

```

git add environments/stage/
git commit -m "feat: add stage environment manifests"
git push origin main

```

Create Promotion Pipeline Resources

Pipeline Workspace

The promotion pipeline needs a workspace for cloning the GitOps repository:

```

oc apply -f - <<'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: promotion-workspace-pvc
  namespace: etx-app-dev
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
EOF

```

Fetch Quay Credentials from Vault

Create a task to retrieve the Quay robot credentials you stored in Vault:

```

oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: vault-fetch-quay-creds
  namespace: etx-app-dev
spec:
  results:
    - name: quay-server
    - name: quay-username
    - name: quay-password
  steps:
    - name: fetch
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      script: |
        #!/bin/sh
        set -e

        # Install jq
        microdnf install -y jq && microdnf clean all

        # Authenticate to Vault using ServiceAccount token
        VAULT_ADDR="{vault_url}"
        SA_TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)

        VAULT_TOKEN=$(curl -s --request POST \
          "${VAULT_ADDR}/v1/auth/kubernetes/login" \
          --data '{"role":"pipeline","jwt":"'${SA_TOKEN}'"}' \
          | jq -r '.auth.client_token')

        # Fetch Quay credentials

```

```

CREDS=$(curl -s --header "X-Vault-Token: ${VAULT_TOKEN}" \
  "${VAULT_ADDR}/v1/secret/data/registry/quay")

echo "${CREDS}" | jq -r '.data.data.server' \
  | tr -d '\n' > $(results.quay-server.path)
echo "${CREDS}" | jq -r '.data.data.username' \
  | tr -d '\n' > $(results.quay-username.path)
echo "${CREDS}" | jq -r '.data.data.password' \
  | tr -d '\n' > $(results.quay-password.path)
EOF

```

Copy Image to Quay

Create a task that copies the image from the internal registry to Quay using Skopeo:

```

oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: copy-image-to-quay
  namespace: etx-app-dev
spec:
  params:
    - name: source-image
      type: string
      description: Source image URL (internal registry)
    - name: dest-image
      type: string
      description: Destination image URL (Quay)
    - name: dest-username
      type: string
    - name: dest-password
      type: string
  steps:
    - name: copy
      image: quay.io/skopeo/stable:latest
      script: |
        #!/bin/bash
        set -e

        echo "Copying image from internal registry to Quay..."
        echo "Source: $(params.source-image)"
        echo "Destination: $(params.dest-image)"

        # Authenticate to the OpenShift internal registry with the
        # pipeline ServiceAccount token (destination uses Quay robot creds)
        SRC_TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)

        # Copy image using skopeo
        skopeo copy \

```

```

--src-tls-verify=false \
--dest-tls-verify=false \
--src-creds="pipeline:${SRC_TOKEN}" \
--dest-creds="$(params.dest-username):$(params.dest-password)" \
"docker://$(params.source-image)" \
"docker://$(params.dest-image)"

```

```
echo "Image copied successfully!"
```

```
EOF
```

Update GitOps Repository

Create a task to update the staging deployment manifest with the new image tag:

```

oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: update-gitops-staging
  namespace: etx-app-dev
spec:
  params:
    - name: gitops-repo-url
      type: string
    - name: image-url
      type: string
      description: Full image URL to update in staging manifest
    - name: git-username
      type: string
      default: "Tekton Pipeline"
    - name: git-email
      type: string
      default: "pipeline@openshift.local"
  workspaces:
    - name: source
  steps:
    - name: update
      image: registry.access.redhat.com/ubi9/ubi:latest
      script: |
        #!/bin/bash
        set -e

        # Install git
        dnf install -y git && dnf clean all

        REPO_URL="$(params.gitops-repo-url)"
        GITLAB_HOST=$(echo "$REPO_URL" | sed -E 's|https?://([^\/]+)/.*|\1|')

        git config --global url."https://etx-platform-
bot:openshift@${GITLAB_HOST}/".insteadOf "https://${GITLAB_HOST}/"

```

```

# Clone GitOps repo
git clone "$REPO_URL" /workspace/gitops
cd /workspace/gitops

# Update the image in staging deployment
sed -i "s|image:.*etx-app-base-app:.*|image: $(params.image-url)|" \
    environments/stage/deployment.yaml

# Commit and push only when the image reference changed
git config user.email "$(params.git-email)"
git config user.name "$(params.git-username)"
git add environments/stage/deployment.yaml
if git diff --cached --quiet; then
    echo "No changes to commit - staging already references $(params.image-url)"
else
    git commit -m "Promote to staging: $(params.image-url)"
    git push origin main
    echo "GitOps repository updated successfully!"
fi
EOF

```

Create the Promotion Pipeline

Now combine all tasks into a promotion pipeline:

```

oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: etx-app-promotion
  namespace: etx-app-dev
spec:
  params:
    - name: source-image-tag
      type: string
      description: Image tag to promote (e.g., commit SHA)
    - name: dest-image-tag
      type: string
      default: stable
      description: Tag to use in Quay (e.g., stable, v1.0.0)
    - name: gitops-repo-url
      type: string
      default: {gitlab_url}/software-factory/etx-app-gitops.git
  workspaces:
    - name: shared-workspace
  tasks:
    - name: fetch-quay-creds
      taskRef:

```

```

    name: vault-fetch-quay-creds

- name: copy-to-quay
  runAfter: [fetch-quay-creds]
  taskRef:
    name: copy-image-to-quay
  params:
    - name: source-image
      value: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-app-
base-app:${params.source-image-tag}
    - name: dest-image
      value: ${tasks.fetch-quay-creds.results.quay-server}/etx-app/etx-app-base-
app:${params.dest-image-tag}
    - name: dest-username
      value: ${tasks.fetch-quay-creds.results.quay-username}
    - name: dest-password
      value: ${tasks.fetch-quay-creds.results.quay-password}

- name: update-gitops
  runAfter: [copy-to-quay]
  taskRef:
    name: update-gitops-staging
  params:
    - name: gitops-repo-url
      value: ${params.gitops-repo-url}
    - name: image-url
      value: ${tasks.fetch-quay-creds.results.quay-server}/etx-app/etx-app-base-
app:${params.dest-image-tag}
  workspaces:
    - name: source
      workspace: shared-workspace
EOF

```

Test the Promotion Pipeline

First, get the GitLab URL dynamically:

```

GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "GitLab URL: $GITLAB_URL"

```

Get a Valid Image Tag

Find a recent image tag from your dev environment:

```

# List recent images from dev CI pipeline
oc get imagestream etx-app-base-app -n etx-app-dev -o json | \

```

```
jq -r '.status.tags[].tag' | head -5
```

Pick one of the commit SHA tags from the output.

Run the Promotion Pipeline

Trigger the promotion pipeline manually.

First, get an actual commit SHA from the images built by the CI pipeline:

```
oc get images -n etx-app-dev | grep etx-app-base-app
```

Then create a PipelineRun (replace `<COMMIT_SHA>` with an actual image tag from above):

```
oc create -f - <<EOF
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: etx-app-promotion-
  namespace: etx-app-dev
spec:
  pipelineRef:
    name: etx-app-promotion
  params:
    - name: source-image-tag
      value: <COMMIT_SHA>
    - name: dest-image-tag
      value: stable
    - name: gitops-repo-url
      value: ${GITLAB_URL}/software-factory/etx-app-gitops.git
  workspaces:
    - name: shared-workspace
      persistentVolumeClaim:
        claimName: promotion-workspace-pvc
  taskRunTemplate:
    serviceAccountName: pipeline
EOF
```

Watch the pipeline logs. You should see:

1. Vault credentials retrieved
2. Image copied from internal registry to Quay
3. GitOps repository updated

Create Stage Application

Create an Argo CD Application for the staging environment on the secondary cluster.

First, get the GitLab URL:

```
# Get GitLab URL
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "Using GitLab URL: $GITLAB_URL"
```

Then create the Application:

```
oc apply -f - <<EOF
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: etx-app-stage
  namespace: etx-app-dev
  labels:
    app.kubernetes.io/part-of: etx_app_base_app
spec:
  project: default
  source:
    repoURL: ${GITLAB_URL}/software-factory/etx-app-gitops.git
    targetRevision: main
    path: environments/stage
  destination:
    name: staging
    namespace: etx-app-staging
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
EOF
```



The staging cluster destination points to the secondary cluster that was pre-configured by the bootstrap automation.

Verify Stage Deployment

Check the stage application status:

```
oc get application etx-app-stage -n etx-app-dev \
-o jsonpath='{.status.sync.status}'{"\n"}
```

The output should be **Synced**.

View the application in Argo CD UI:

1. Open the Argo CD dashboard
2. You should see two applications:
 - **etx-app-dev** - Running on primary cluster
 - **etx-app-stage** - Running on secondary (staging) cluster
3. Click on **etx-app-stage** to view its deployment status



Since Argo CD is managing the staging cluster remotely, you can view the application status through the Argo CD UI without needing to login to the secondary cluster directly.

Promotion Workflow Overview

The promotion workflow from dev to staging follows these steps:

1. CI pipeline builds and pushes image to internal registry with commit SHA tag
2. Image is promoted to Quay registry with a **stable** tag (covered in [Promotion Pipeline](#))
3. GitOps repository **environments/stage/deployment.yaml** is updated with new image tag
4. Argo CD detects the change and automatically syncs to the secondary cluster

This GitOps approach ensures:

- **Version control** - All deployment changes are tracked in Git
- **Auditability** - Full history of what was deployed when
- **Rollback capability** - Revert to previous commit to rollback
- **Multi-cluster consistency** - Same manifests can deploy to multiple clusters

Quick Verification

Verify the complete deployment:

```
# Check Argo CD application status
oc get application etx-app-dev -n etx-app-dev -o jsonpath='{.status.sync.status}:{.status.health.status}{"\n"}'

# Check application pods
oc get pods -n etx-app-dev -l app=etx-app-base-app

# Check application health endpoint
curl https://$(oc get route etx-app-base-app -n etx-app-dev -o jsonpath='{.spec.host}')/q/health

# Check OpenTelemetry injection
oc get pod -n etx-app-dev -l app=etx-app-base-app \
```

```
-o jsonpath='{.items[0].spec.initContainers[*].name}{"\n"}'
```

Expected output should show: **Synced: Healthy** for Argo CD, **Running** pods, and **opentelemetry-auto-instrumentation** init container.



Common Issues:

- **ImagePullBackOff:** Image not in registry - run CI pipeline first
- **CrashLoopBackOff:** Check pod logs - often missing Kafka or DB connection
- **Argo CD Out of Sync:** Click "Sync" in Argo CD UI or check GitOps repo for errors
- **Health check failing:** Verify all dependencies (DB, Kafka) are running

Verify the Promotion

Check that the GitOps repository was updated:

```
cd ~/workshop/etx-app-gitops
git pull origin main
cat environments/stage/deployment.yaml | grep image:
```

You should see the Quay image URL with the **stable** tag.

Check the Argo CD application status:

```
oc get application etx-app-stage -n etx-app-dev -o
jsonpath='{.status.sync.status}{"\n"}'
```

The output should be **Synced**.

View in Argo CD UI:

1. Open the Argo CD dashboard
2. Click on the **etx-app-stage** application
3. You should see it's synced and healthy
4. The deployment should show the new image from Quay

Automate Promotion with Approval

For a more realistic workflow, you can add a manual approval step before promotion.

Option 1: Manual Trigger (Current Approach)

The current setup requires manually triggering the promotion pipeline. This is suitable for:

- Small teams
- Infrequent deployments
- High-touch release processes

Option 2: Automatic with Manual Approval

Add a manual approval task using Tekton:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: manual-approval
  namespace: etx-app-dev
spec:
  description: |
    Pauses pipeline execution until manual approval.
    Requires operator intervention to continue.
  steps:
    - name: wait-for-approval
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      script: |
        #!/bin/sh
        echo "" Pipeline paused - waiting for manual approval"
        echo "Run: oc patch taskrun <taskrun-name> -p
'{"spec\":{\"status\":"Succeeded\"}}'"
        sleep infinity
EOF
```

Add this task as the first step in your promotion pipeline to require approval before promoting to staging.

Option 3: GitLab Merge Request Trigger

Create a webhook that triggers the promotion pipeline when a merge request is merged:

1. Create a promotion branch in your application repo
2. Open a merge request for promotion
3. Add reviewers for approval
4. On merge, trigger the promotion pipeline via GitLab webhook

This approach provides:

- Code review for promotion decisions
- Audit trail in Git
- Team collaboration

Rollback Strategy

If you need to rollback a staging deployment:

Method 1: Revert Git Commit

```
cd ~/workshop/etx-app-gitops
git log environments/stage/deployment.yaml
git revert <commit-hash>
git push origin main
```

Argo CD will automatically sync the previous version.

Method 2: Update to Previous Tag

Run the promotion pipeline with a previous image tag:

```
oc create -f - <<EOF
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: etx-app-promotion-
  namespace: etx-app-dev
spec:
  pipelineRef:
    name: etx-app-promotion
  params:
    - name: source-image-tag
      value: <PREVIOUS_SHA>
    - name: dest-image-tag
      value: stable
  workspaces:
    - name: shared-workspace
      persistentVolumeClaim:
        claimName: promotion-workspace-pvc
  taskRunTemplate:
    serviceAccountName: pipeline
EOF
```

Summary

You have completed the Promotion Pipeline exercise. You now have:

- Automated image promotion from internal registry to Quay
- GitOps-based staging deployments
- Secure credential management via Vault

- Multi-cluster deployment capability
- Rollback capability through Git history

Key Takeaways

Promotion != Re-build:

- The same container image (by SHA) is promoted from dev to staging
- No code changes between environments - only configuration
- This ensures you test exactly what you deploy

GitOps ensures consistency:

- All deployment state is in Git
- Changes are auditable and reversible
- Argo CD provides automatic sync and drift detection

Multi-cluster architecture:

- Dev uses internal registry (fast, local)
- Staging uses Quay (cross-cluster accessible)
- Argo CD manages both from single control plane